UNIT-4 (TREES)

BINARY TREE – TERMINOLOGY AND PROPERTIES

LECTURE NO:-26

Tree: A tree is a finite set of one or more nodes such that, there is a specially designated node called root. The remaining nodes are partitioned into n>=0 disjoint sets T1, T2,..Tn, where each of these set is a tree T1,…Tn are called the subtrees of the root.

Branch: Branch is the link between the parent and its child

Leaf: A node with no children is called a leaf.

Subtree: A Subtree is a subset of a tree that is itself a tree.

Degree: The number of subtrees of a node is called the degree of the node. Hence nodes that have degree zero are called leaf or terminal nodes. The other nodes are referred as non-terminal nodes.

Children: The nodes branching from a particular node X are called children of X and X is called its parent.

Siblings: Children of the same parent are said to be siblings

Degree of tree: Degree of the tree is the maximum of the degree of the nodes in the tree.

Ancestors: Ancestors of a node are all the nodes along the path from root to that node. Hence root is ancestor of all the nodes in the tree.

Level: Level of a node is defined by letting root at level one. If a node is at level L, then its children are at level L + 1.

Height or depth: The height or depth of a tree is defined to be the maximum level of any node in the tree.

Climbing: The process of traversing the tree from the leaf to the root is called climbing the tree.

Descending: The process of traversing the tree from the root to the leaf is called descending the tree.

Binary tree: A binary tree is a finite set of nodes that either is empty or consists of a root and two disjoint binary trees called the left subtree and right subtree.

Left child: The node present to the left of the parent node is called the left child.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

MITRC

Right child: The node present to the right of the parent node is called the right child

Skewed Binary tree: If the new nodes in the tree are added only to one side of the binary tree then it is a skewed binary tree.

Strictly binary tree: If the binary tree has each node consisting of either two nodes or no nodes at all, then it is called a strictly binary tree.

Complete binary tree: If all the nodes of a binary tree consist of two nodes each and the nodes at the last level does not consist any nodes, then that type of binary tree is called a complete binary tree.

PROPERTIES OF BINARY TREES

- The number of nodes n in a full binary tree, is at least n= 2h+1 and at most n = 2 h+1-1, where h is the height of the tree.
- A binary tree of n elements has n-1 edges.
- A binary tree of height h has at least h and at most 2h - 1 elements
- A tree consisting of only a root node has a height of 0.
- The number of leaf nodes in a perfect binary tree, is l= (n+1)/2. This means that a perfect binary tree with l leaves has n=2l-1 nodes.
- The number of internal nodes in a complete binary tree of n nodes is n/2. REPRESENTATION OF BINARY TREES There are two ways in which a binary tree can be represented. They are:
  (i) Array representation of binary trees.
  (ii) Linked representation of binary trees.

MITRC

UNIT-4 (TREES)

TREE TRAVERSALS

LECTURE NO:-27

BINARY TREE TRAVERSALS

There are three standard ways of traversing a binary tree T with root R. They are:

(i) Preorder Traversal

(ii) Inorder Traversal

(iii) Postorder Traversal

General outline of these three traversal methods can be given as follows:

Preorder Traversal:

(1) Process the root R.

(2) Traverse the left subtree of R in preorder.

(3) Traverse the right subtree of R in preorder.

Inorder Traversal:

(1) Traverse the left subtree of R in inorder.

(2) Process the root R.

(3) Traverse the right subtree of R in inorder.

Postorder Traversal:

(1) Traverse the left subtree of R in postorder.

(2) Traverse the right subtree of R in postorder.

(3) Process the root R.

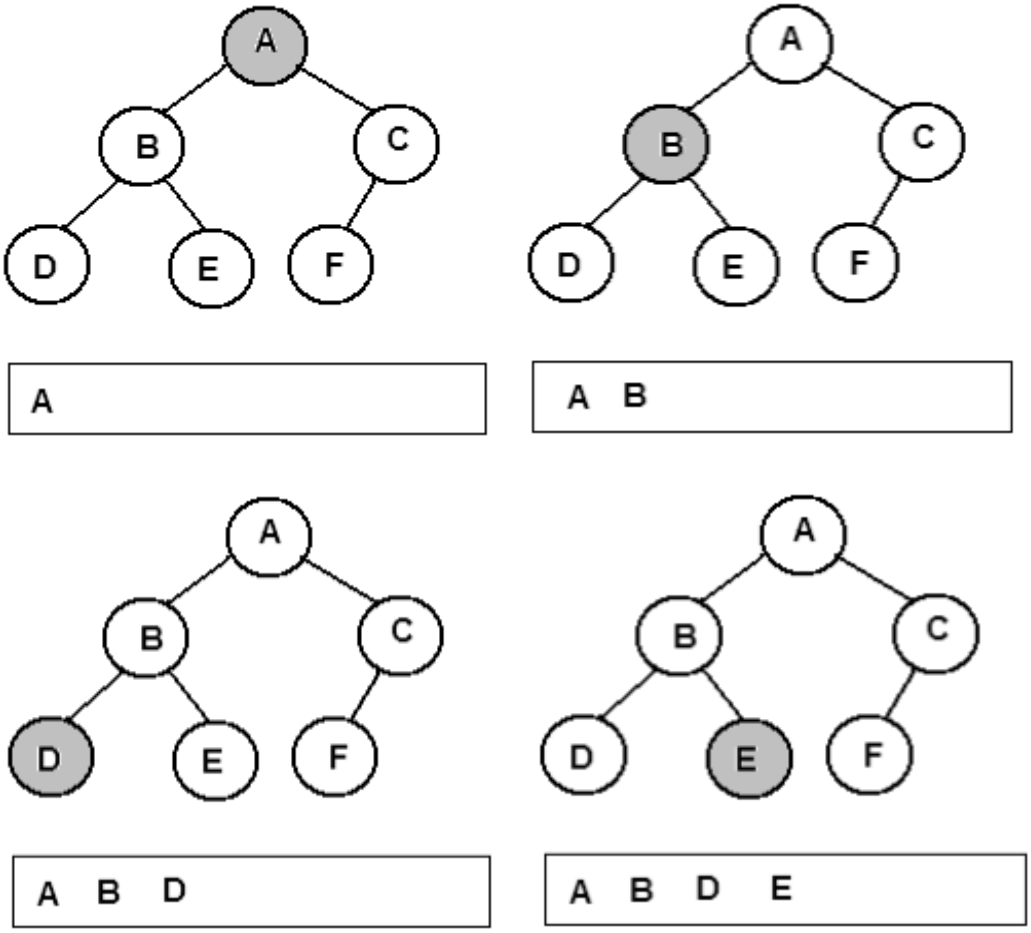
MITRC

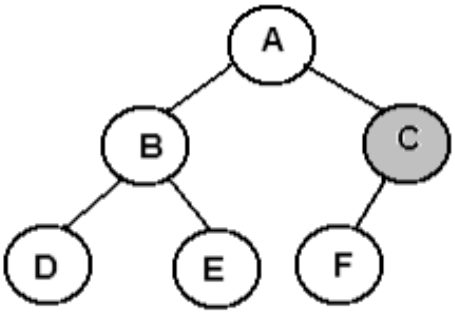
Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

Traversal algorithms using recursive approach

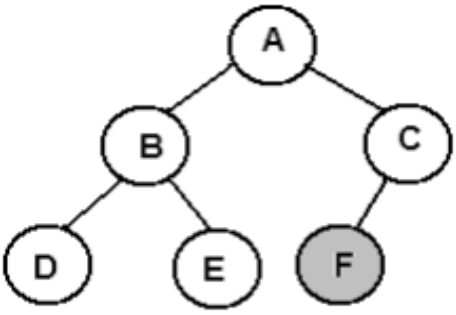
Preorder Traversal

In the preorder traversal the node element is visited first and then the right subtree of the node and then the right subtree of the node is visited. Consider the following case where we have 6 nodes in the tree A, B, C, D, E, F. The traversal always starts from the root of the tree. The node A is the root and hence it is visited first. The value at this node is processed. The processing can be doing some computation over it or just printing its value. Now we check if there exists any left child for this node if so apply the preorder procedure on the left subtree. Now check if there is any right subtree for the node A, the preorder procedure is applied on the right subtree. Since there exists a left subtree for node A, B is now considered as the root of the left subtree of A and preorder procedure is applied. Hence we find that B is processed next and then it is checked if B has a left subtree. This recursive method is continued until all the nodes are visited.





A B D E C



A B D E C F

The algorithm for the above method is presented in the pseudo-code form below:

Algorithm

```
PREORDER( ROOT )

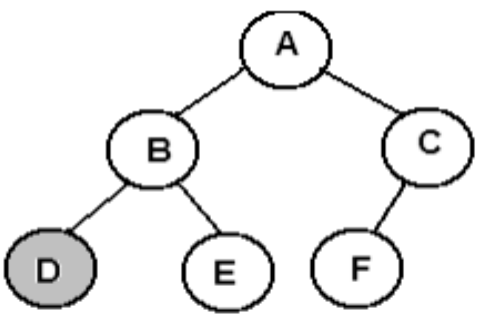
Temp = ROOT
If temp = NULL
        Return
End if
```

```
Print info(temp)
If left(temp) ≠ NULL
        PREORDER( left(temp))
End if
If right(temp) ≠ NULL
        PREORDER(right(temp))
End if
End PREORDER
```

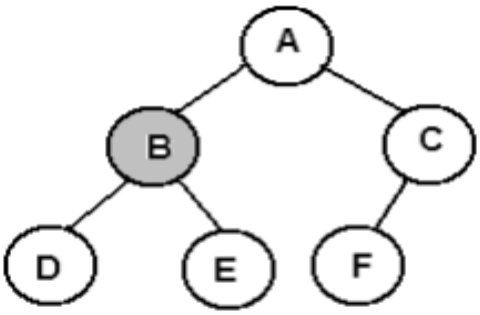
Inorder Traversal

In the Inorder traversal method, the left subtree of the current node is visited first and then the current node is processed and at last the right subtree of the current node is visited. In the following example, the traversal starts with the root of the binary tree. The node A is the root and it is checked if it has the left subtree. Then the inorder traversal procedure is applied on the left subtree of the node A. Now we find that node D does not have left subtree. Hence the node D is processed and then it is checked if there is a right subtree for node D. Since there is no right subtree, the control returns back to the previous function which was applied on B. Since left of B is already visited, now B is processed. It is checked if B has the right subtree. If so apply the

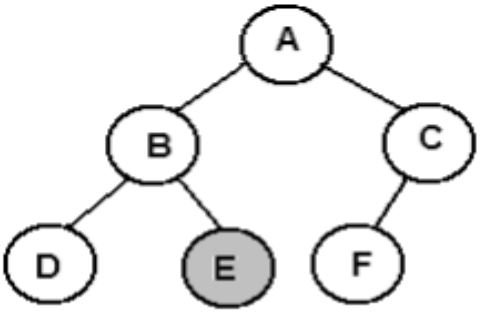
inroder traversal method on the right subtree of the node B. This recursive procedure is followed till all the nodes are visited.



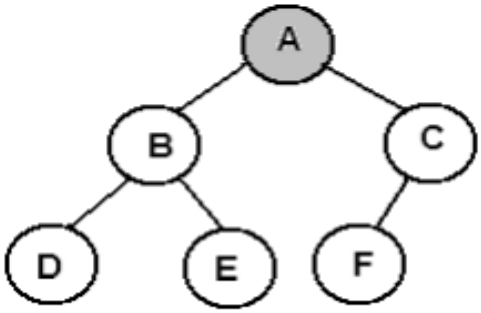
D



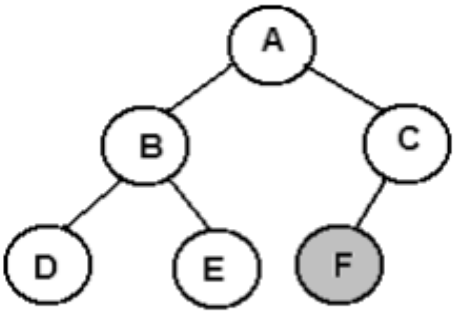
D B



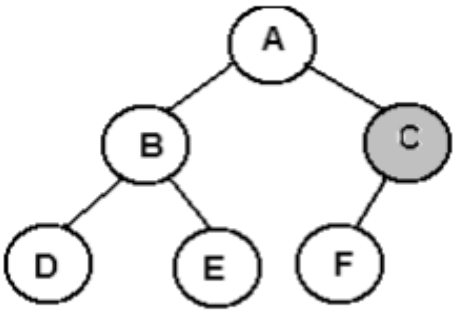
D B E



D B E A



D B E A F



D B E A F C

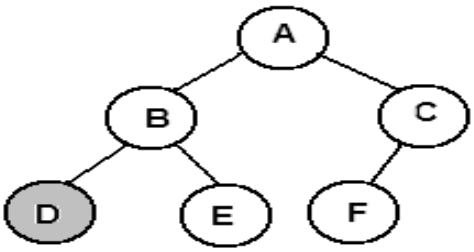
Algorithm

```
INORDER( ROOT )

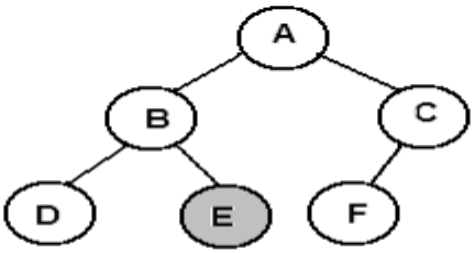
Temp = ROOT
If temp = NULL
        Return
End if
If left(temp) ≠ NULL
        INORDER(left(temp))
End if
Print info(temp)
If right(temp) ≠ NULL
        INORDER(right(temp))
End if
End INORDER
```

Postorder Traversal

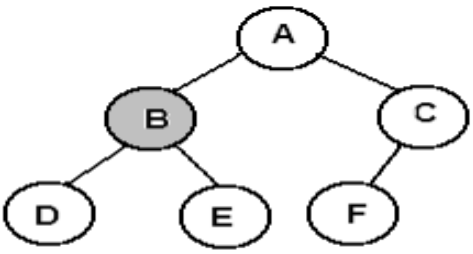
In the postorder traversal method the left subtree is visited first, then the right subtree and at last the current node is processed. In the following example, A is the root node. Since A has the left subtree the postorder traversal method is applied recursively on the left subtree of A. Then when left subtree of A is completely is processed, the postorder traversal method is recursively applied on the right subtree of the node A. If right subtree is completely processed, then the current node A is processed.



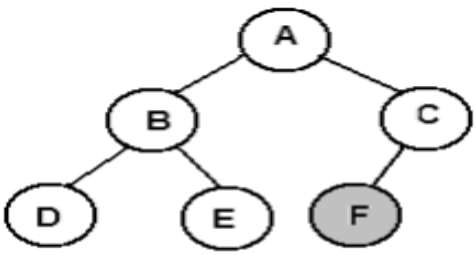
D



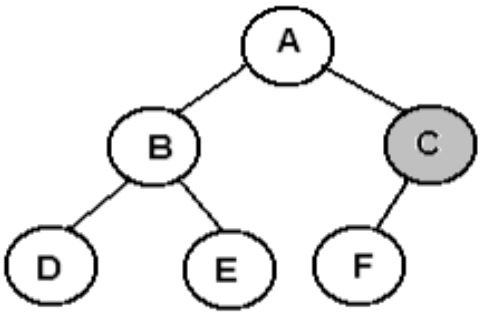
D E



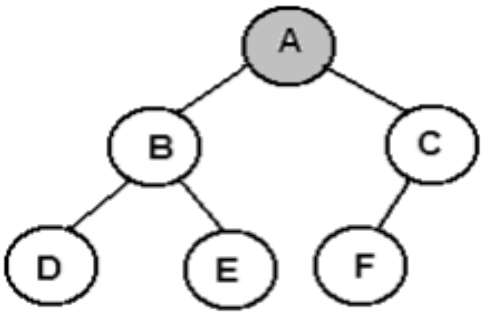
D E B



D E B F



D E B F C



D E B F C A

Algorithm

```
POSTORDER( ROOT )

Temp = ROOT
If temp = NULL
        Return
End if
If left(temp) ≠ NULL
        POSTORDER(left(temp))
```

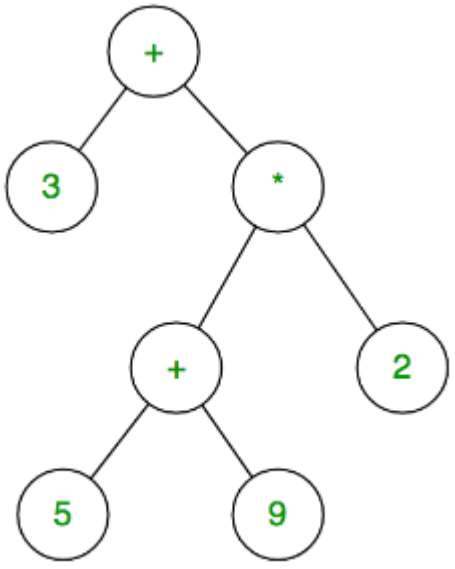
```
End if
If right(temp) ≠ NULL
        POSTORDER(right(temp))
End if
Print info(temp)
End POSTORDER
```

UNIT-4 (TREES)

EXPRESSION TREES

LECTURE NO:-28

The expression tree is a binary tree in which each internal node corresponds to the operator and each leaf node corresponds to the operand so for example expression tree for 3 + ((5+9)*2) would be:



Construction of Expression Tree

Let us consider a postfix expression is given as an input for constructing an expression tree. Following are the step to construct an expression tree:

1. Read one symbol at a time from the postfix expression.
2. Check if the symbol is an operand or operator.
3. If the symbol is an operand, create a one node tree and push a pointer onto a stack
4. If the symbol is an operator, pop two pointers from the stack namely T1 & T2 and form a new tree with root as the operator, T1 & T2 as a left and right child
5. A pointer to this new tree is pushed onto the stack

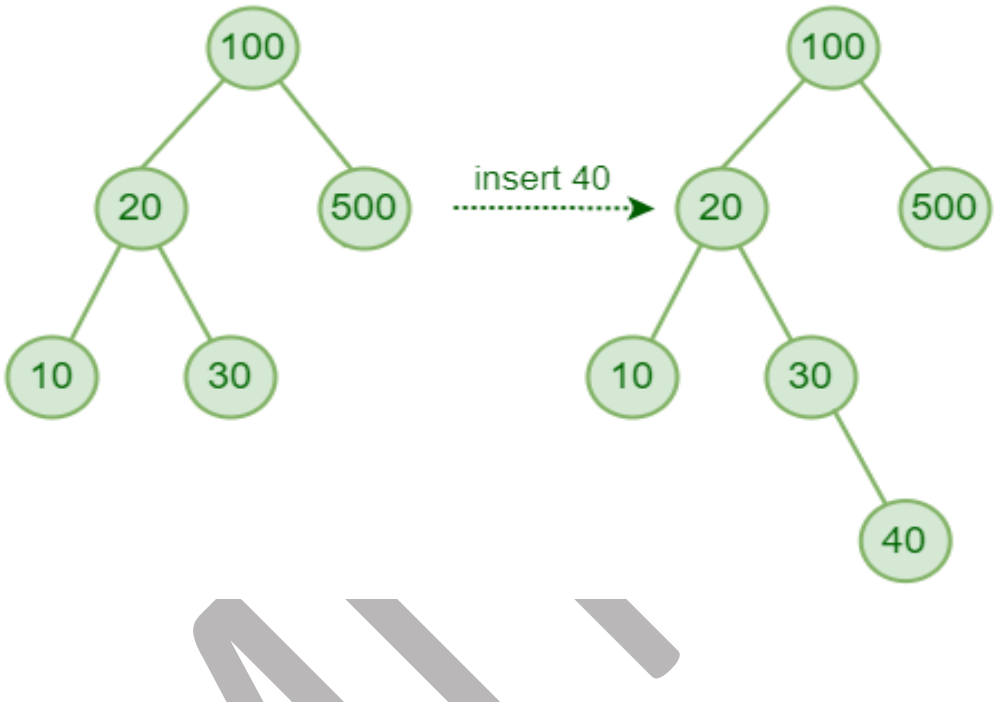
Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

UNIT-4 (TREES)

BINARY SEARCH TREES – OPERATIONS IN BST – INSERTION

LECTURE NO:-29

Given a **BST**, the task is to insert a new node in this **BST**.

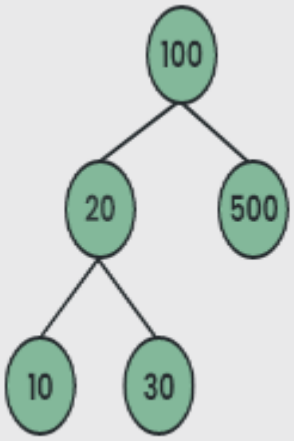


How to Insert a value in a Binary Search Tree:

A new key is always inserted at the leaf by maintaining the property of the binary search tree. We start searching for a key from the root until we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node. The below steps are followed while we try to insert a node into a binary search tree:

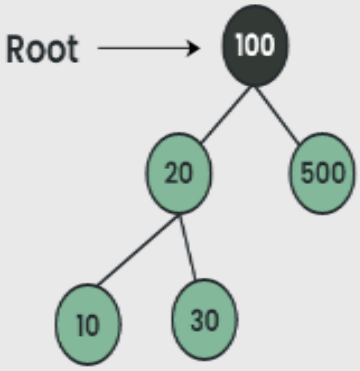
- Check the value to be inserted (say **X**) with the value of the current node (say **val**) we are in:
    - If **X** is less than **val** move to the left subtree.
    - Otherwise, move to the right subtree.
- Once the leaf node is reached, insert **X** to its right or left based on the relation between **X** and the leaf node's value.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)



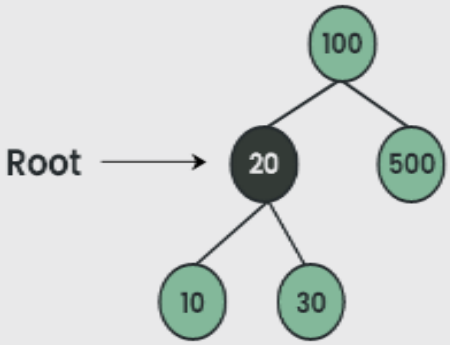
X = 40 (The Node To Be Inserted)

STEP 1 : Comparing X with Root Node

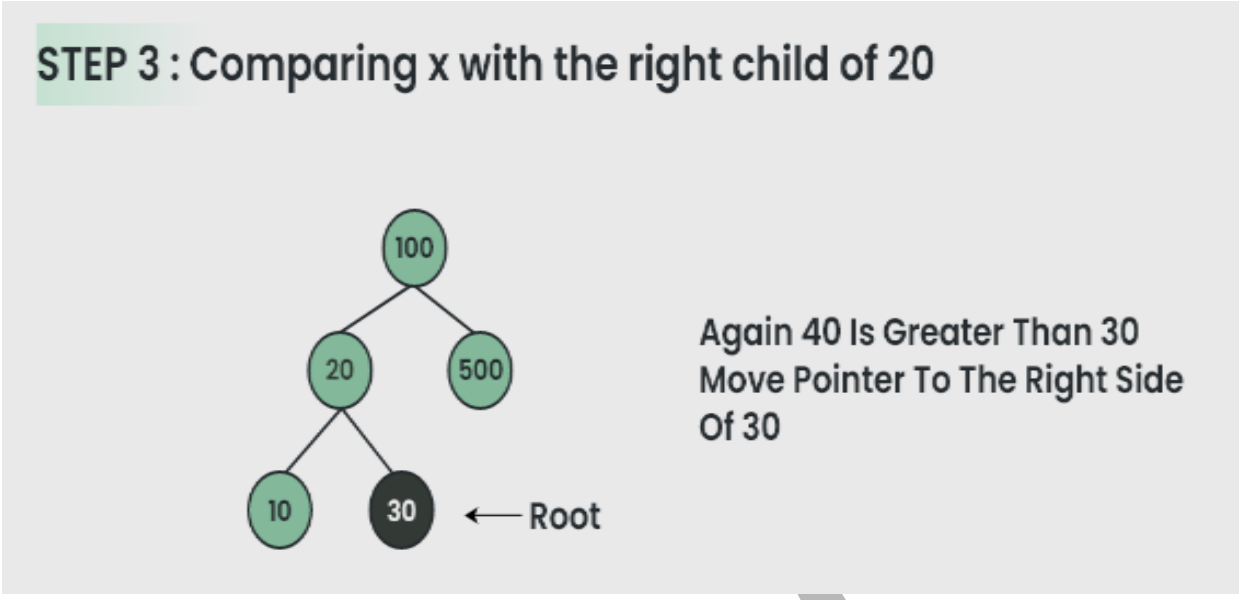


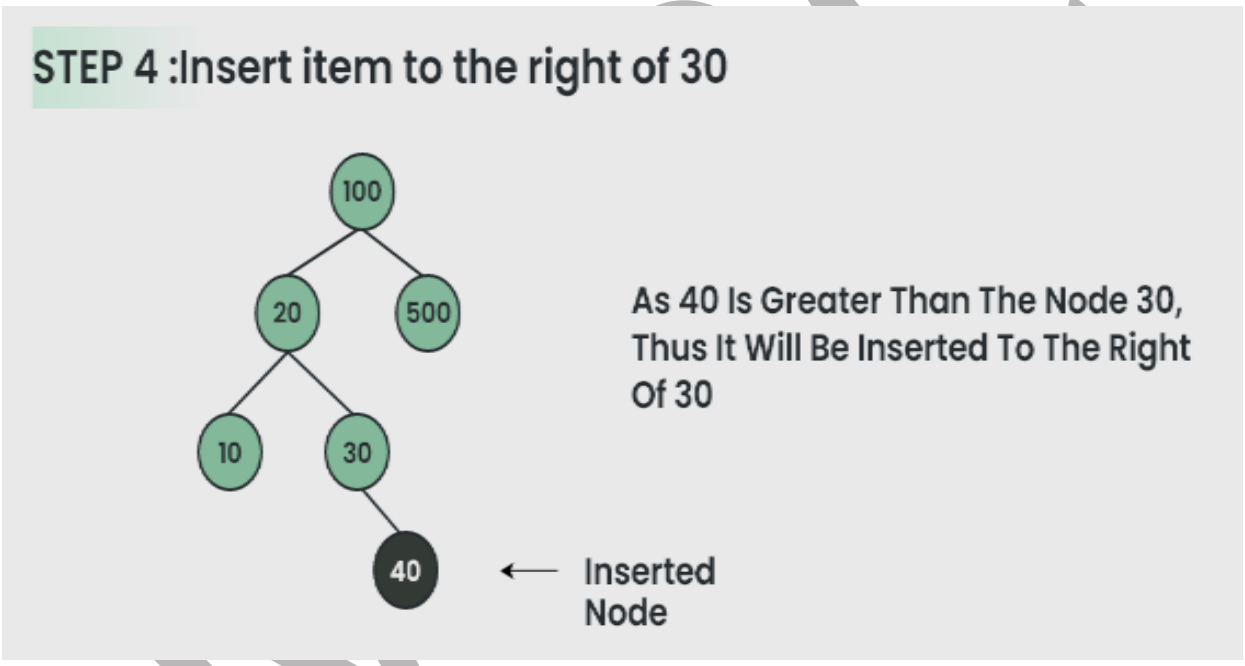
Since 100 Is Greater Than 40.
Move Pointer To The Left Child (20)

STEP 2 : Comparing X with left child of root node



Since 20 Is Less Than 40, Move
Pointer To The Right Child (30)





1. Create a new BST node and assign values to it.

2. insert(node, key)

   i) If root == NULL,

      return the new node to the calling function.

   ii) if root=>data < key

      call the insert function with root=>right and assign the return value in root=>right.

      root->right = insert(root=>right,key)

   iii) if root=>data > key

call the insert function with root->left and assign the return value in root=>left.

root=>left = insert(root=>left,key)

3. Finally, return the original root pointer to the calling function.

MITRC

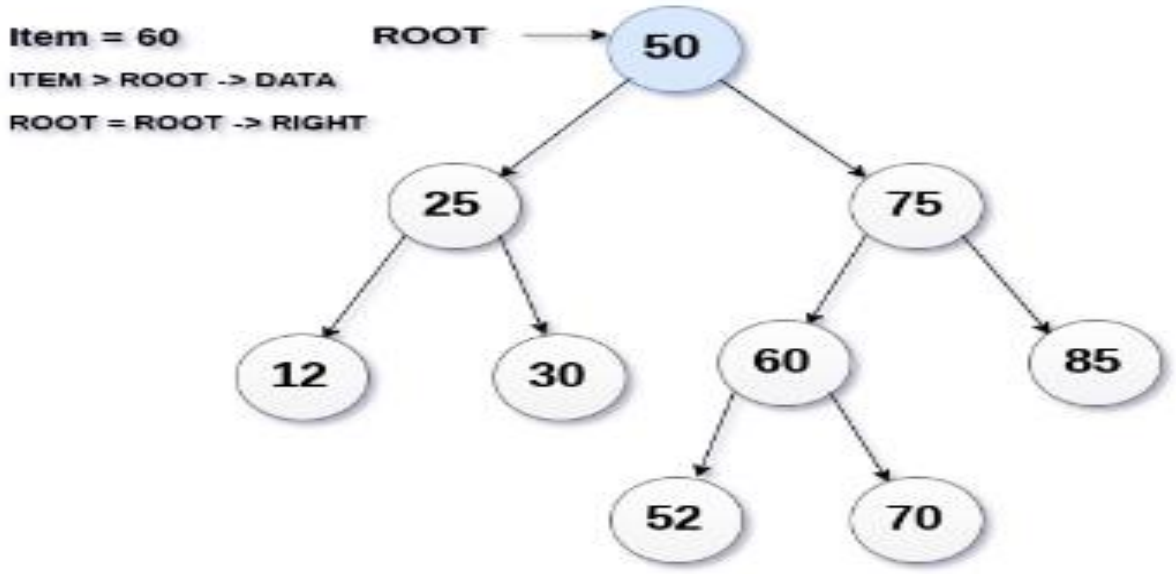
UNIT-4 (TREES)

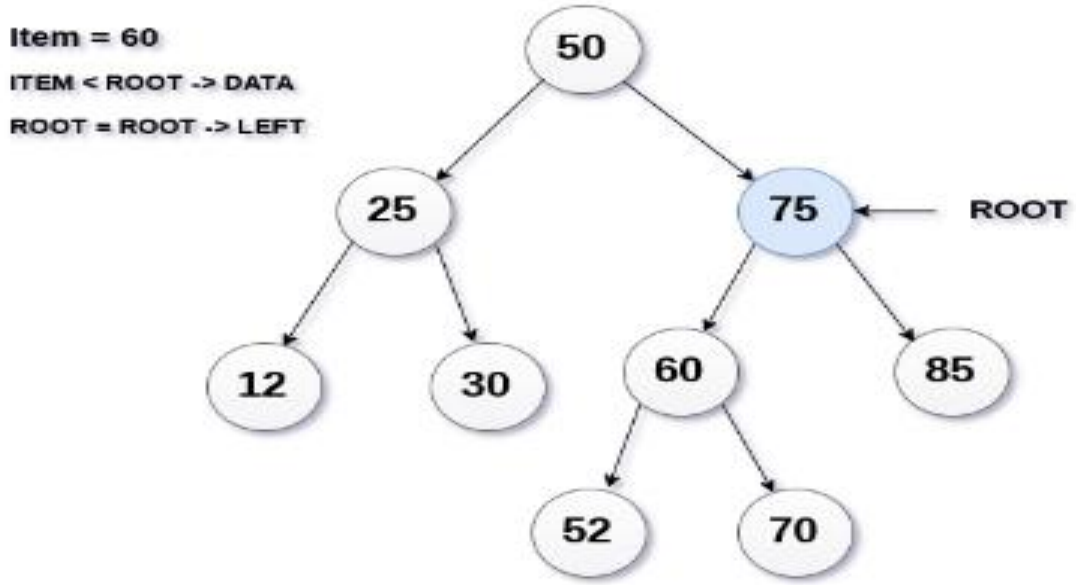
BST SEARCHING, DELETION

LECTURE NO:-30

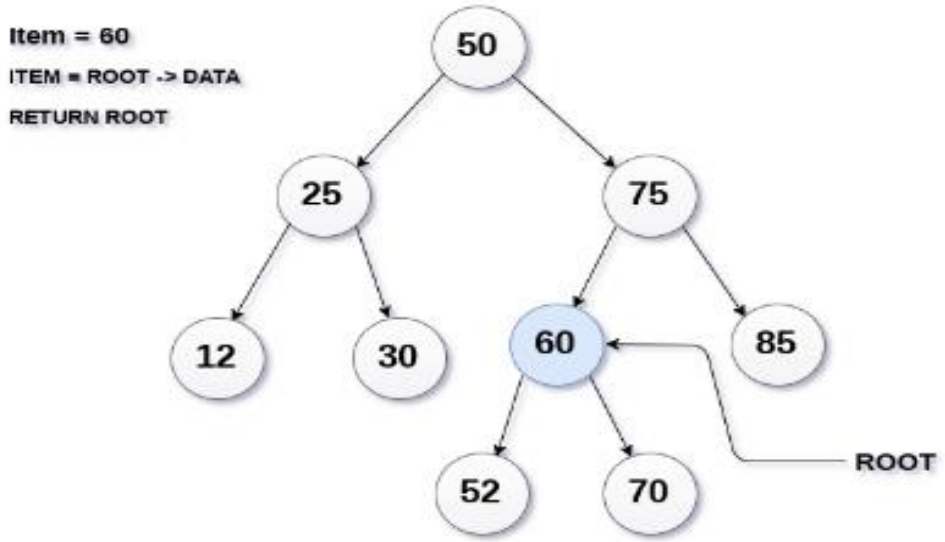
Searching means finding or locating some specific element or node within a data structure. However, searching for some specific node in binary search tree is pretty easy due to the fact that, element in BST are stored in a particular order.

1. Compare the element with the root of the tree.
2. If the item is matched then return the location of the node.
3. Otherwise check if item is less than the element present on root, if so then move to the left sub-tree.
4. If not, then move to the right sub-tree.
5. Repeat this procedure recursively until match found.
6. If element is not found then return NULL.



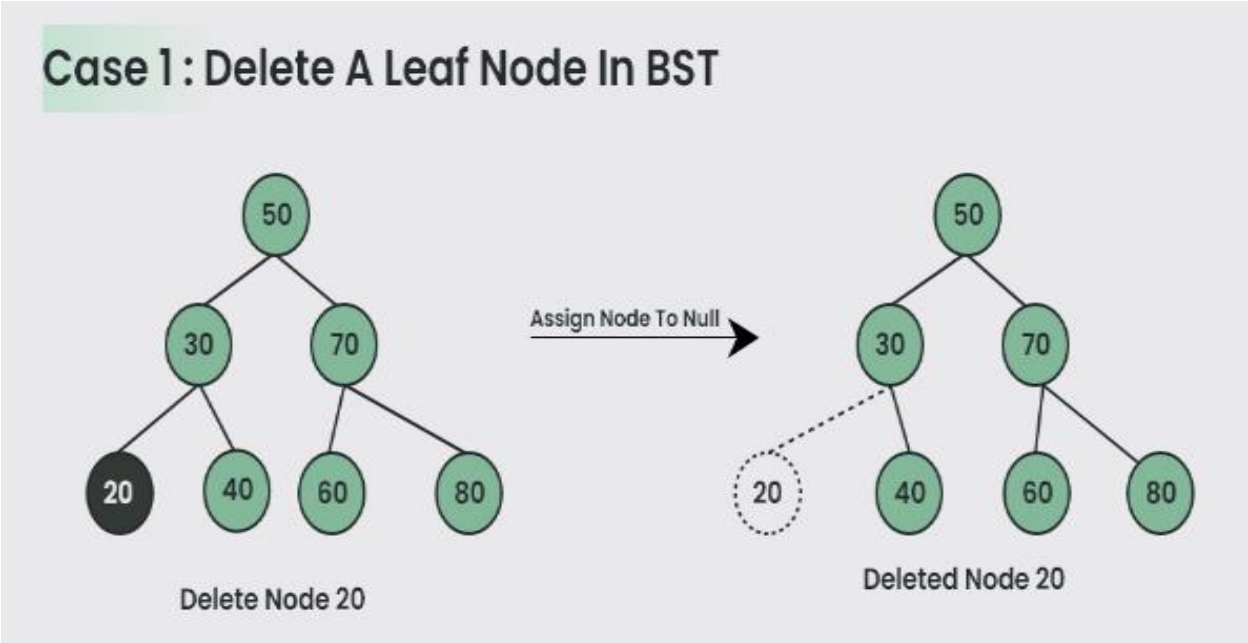

Item = 60
ITEM < ROOT -> DATA
ROOT = ROOT -> LEFT
50
25
75
ROOT
12
30
60
85
52
70

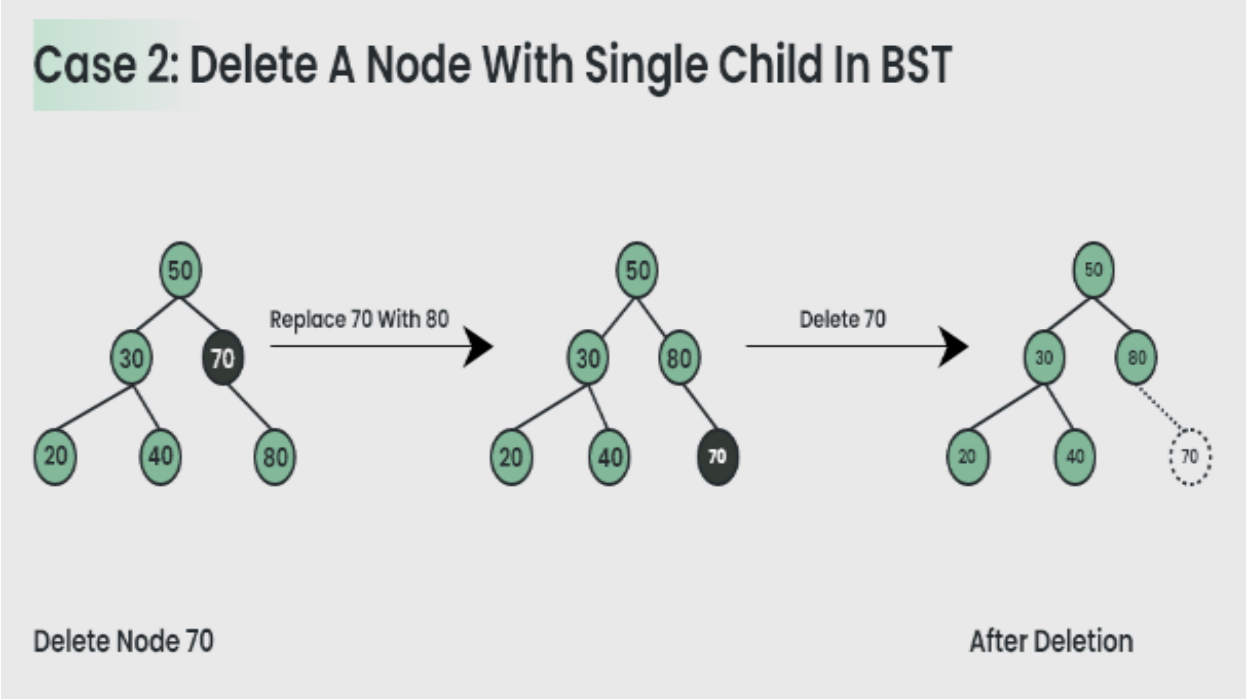
STEP 2

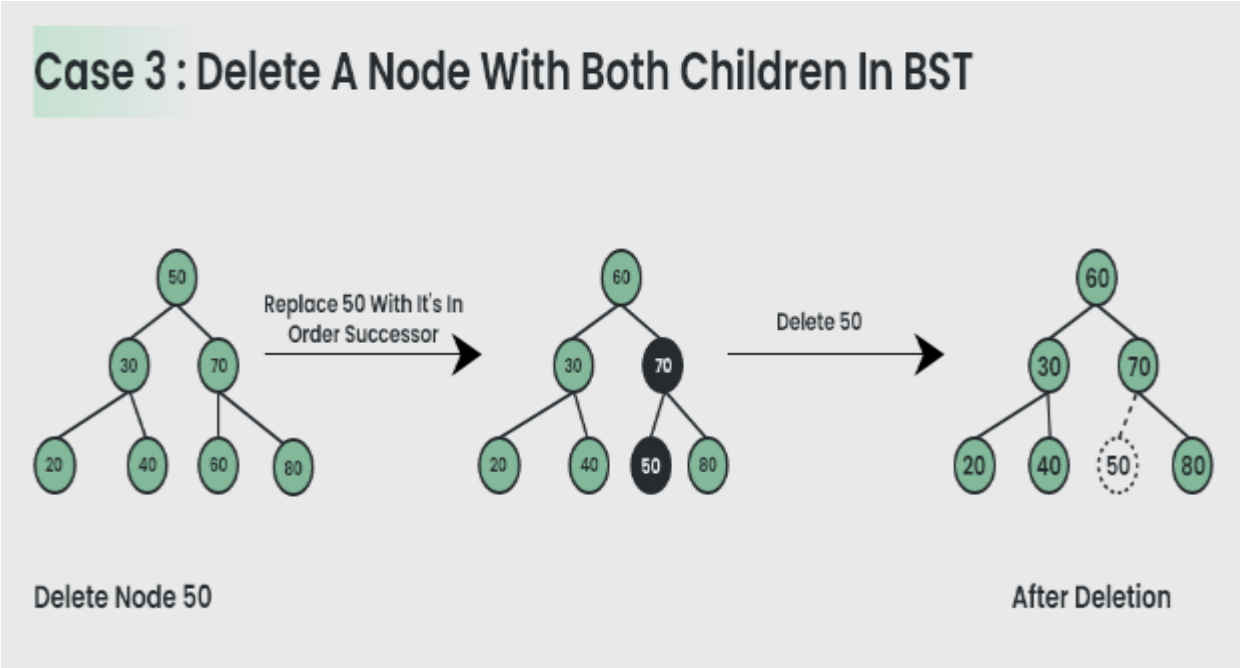

Item = 60
ITEM = ROOT -> DATA
RETURN ROOT
50
25
75
12
30
60
85
ROOT
52
70

STEP 3

Given a **BST**, the task is to delete a node in this **BST**, which can be broken down into 3 scenarios:







Algorithm for Deletion in a Binary Search Tree

1. Start at the root of the binary search tree.
2. If the root is NULL, return NULL.
3. If the value to be deleted is less than the root's data, recursively call the delete function on the left subtree.
4. If the value to be deleted is greater than the root's data, recursively call the delete function on the right subtree.
5. If the value to be deleted is equal to the root's data, check if the node to be deleted has one child or no child.
6. If the node has only one child or no child, free the memory allocated to the node and return the appropriate child (if it exists).
7. If the node to be deleted has two children, find the minimum value in the right subtree (i.e., the leftmost node in the right subtree). Copy the data in the minimum node to the node to be deleted, and recursively call the delete function on the right subtree to delete the minimum node.
8. Return the root of the updated binary search tree.

UNIT-4 (TREES)

AVL TREES-INSERTION, ROTATION IN AVL TREES

LECTURE NO:-31

AVL tree is a self-balancing Binary Search Tree (**BST**) where the difference between heights of left and right subtrees cannot be more than **one** for all nodes.

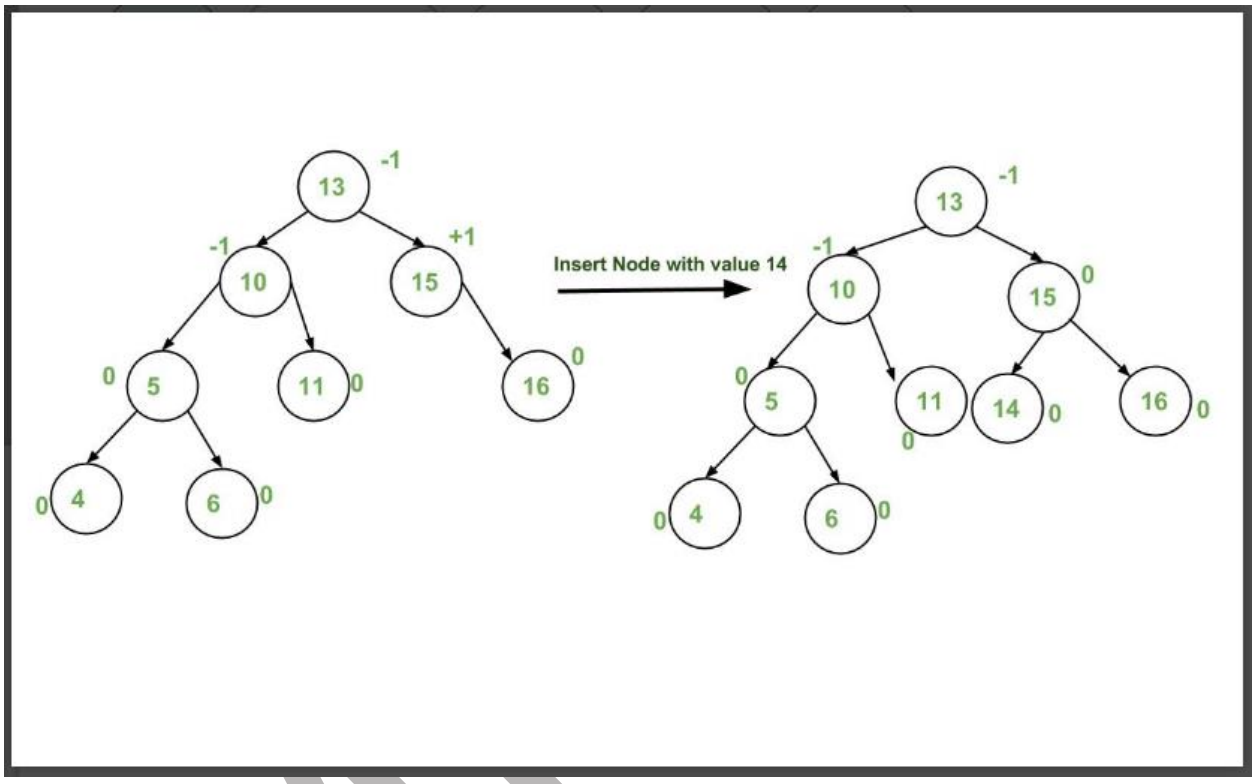
Steps to follow for insertion:

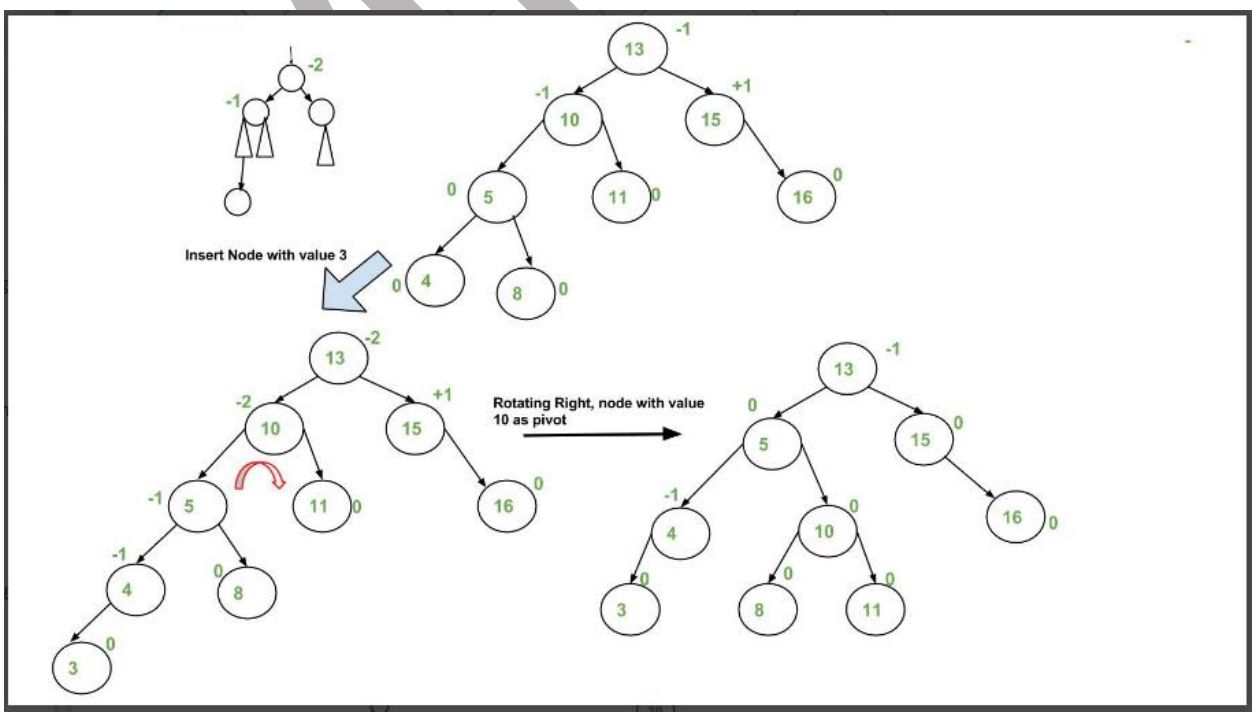
Let the newly inserted node be **w**

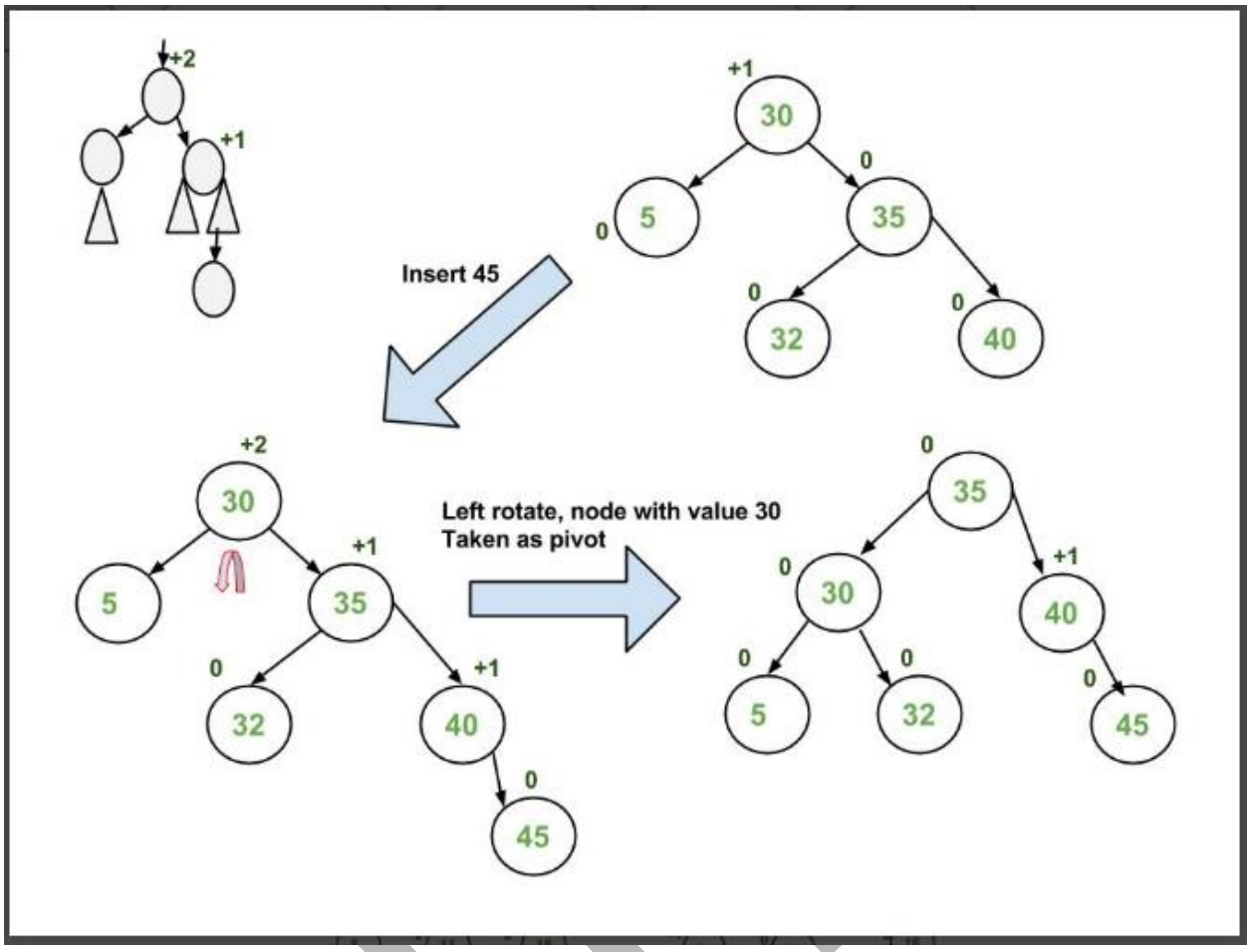
- Perform standard **BST** insert for **w**.
- Starting from **w**, travel up and find the first **unbalanced node**. Let **z** be the first unbalanced node, **y** be the **child** of **z** that comes on the path from **w** to **z** and **x** be the **grandchild** of **z** that comes on the path from **w** to **z**.
- Re-balance the tree by performing appropriate rotations on the subtree rooted with **z.** There can be 4 possible cases that need to be handled as **x, y** and **z** can be arranged in 4 ways.
- Following are the possible 4 arrangements:
    - y is the left child of z and x is the left child of y (Left Left Case)
    - y is the left child of z and x is the right child of y (Left Right Case)
    - y is the right child of z and x is the right child of y (Right Right Case)
    - y is the right child of z and x is the left child of y (Right Left Case)

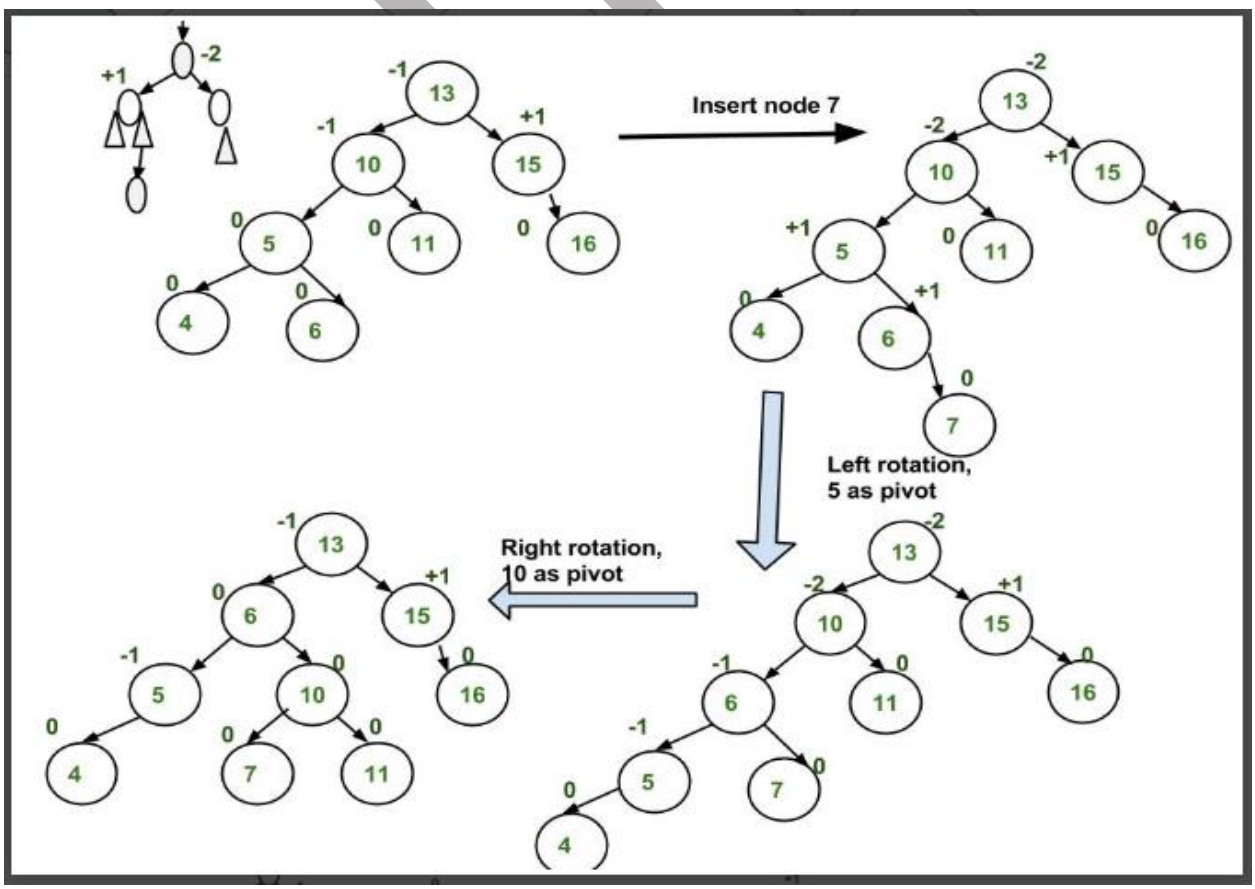
SN	Rotation	Description
1	LL Rotation	The new node is inserted to the left sub-tree of left sub-tree of critical node.
2	RR Rotation	The new node is inserted to the right sub-tree of the right sub-tree of the critical node.
3	LR Rotation	The new node is inserted to the right sub-tree of the left sub-tree of the

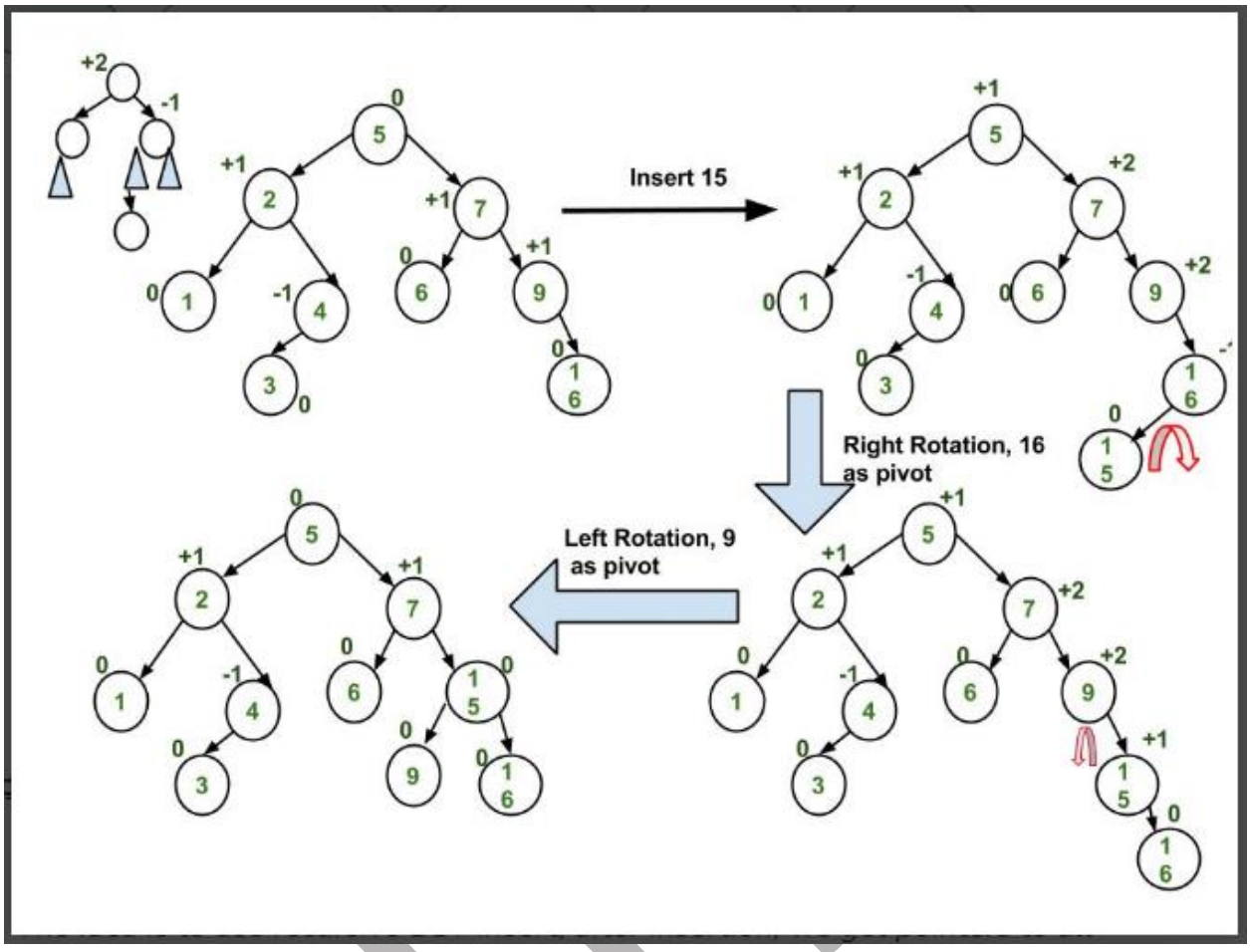
		critical node.
4	RL Rotation	The new node is inserted to the left sub-tree of the right sub-tree of the critical node.






Insert 45
Left rotate, node with value 30
Taken as pivot


Insert node 7
Left rotation,
5 as pivot
Right rotation,
10 as pivot


Insert 15
Right Rotation, 16
as pivot
Left Rotation, 9
as pivot

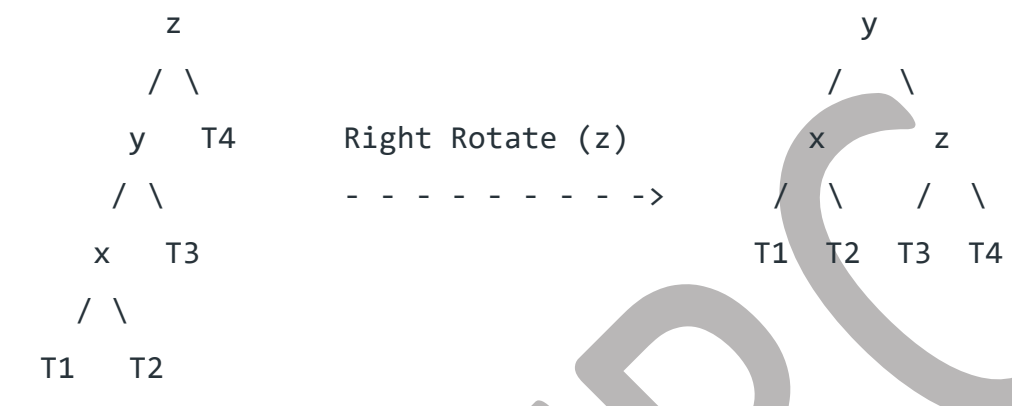
UNIT-4 (TREES)

AVL TREES-DELETION

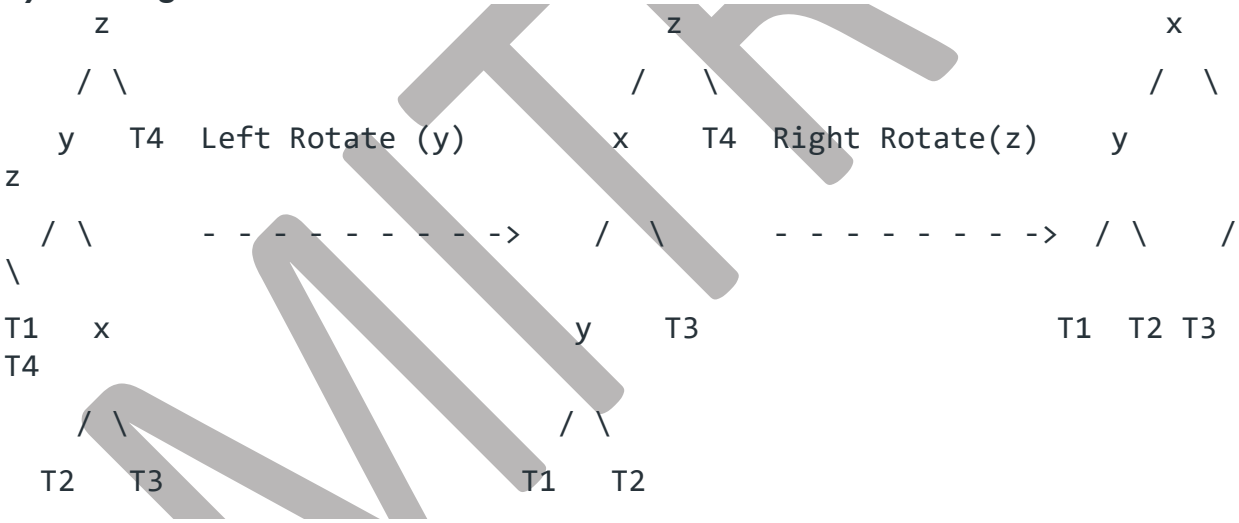
LECTURE NO:-32

a) Left Left Case

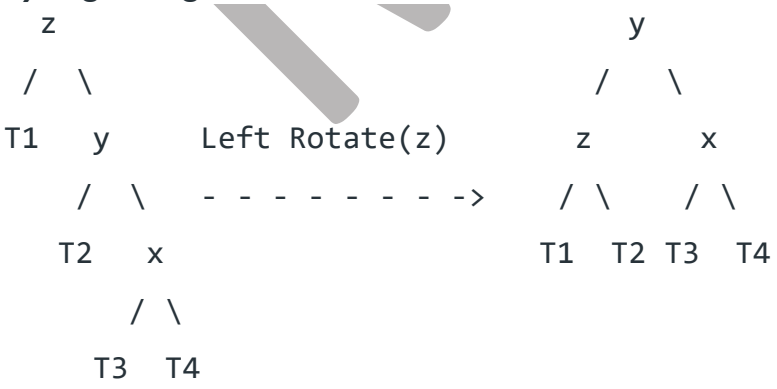
T1, T2, T3 and T4 are subtrees.



b) Left Right Case

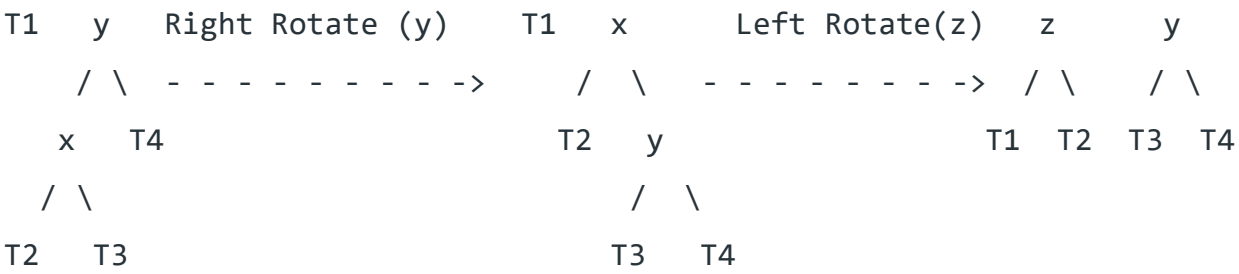


c) Right Right Case



d) Right Left Case



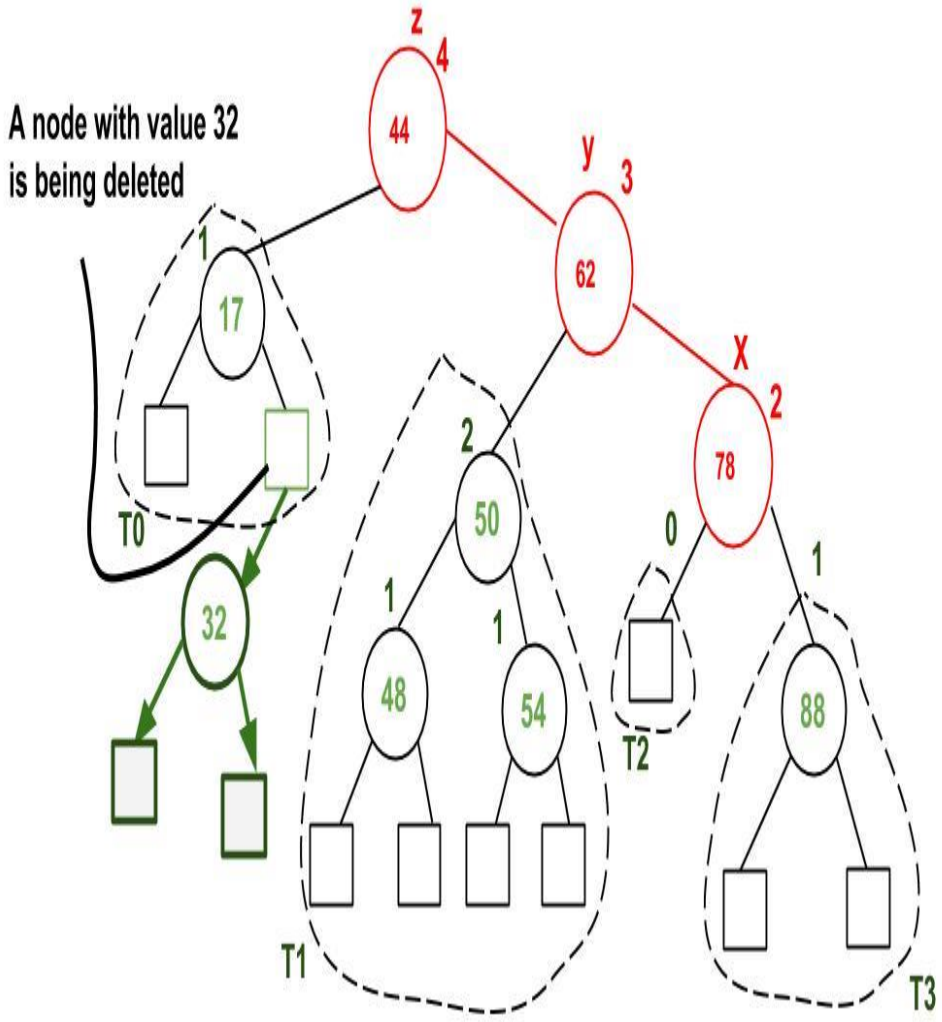


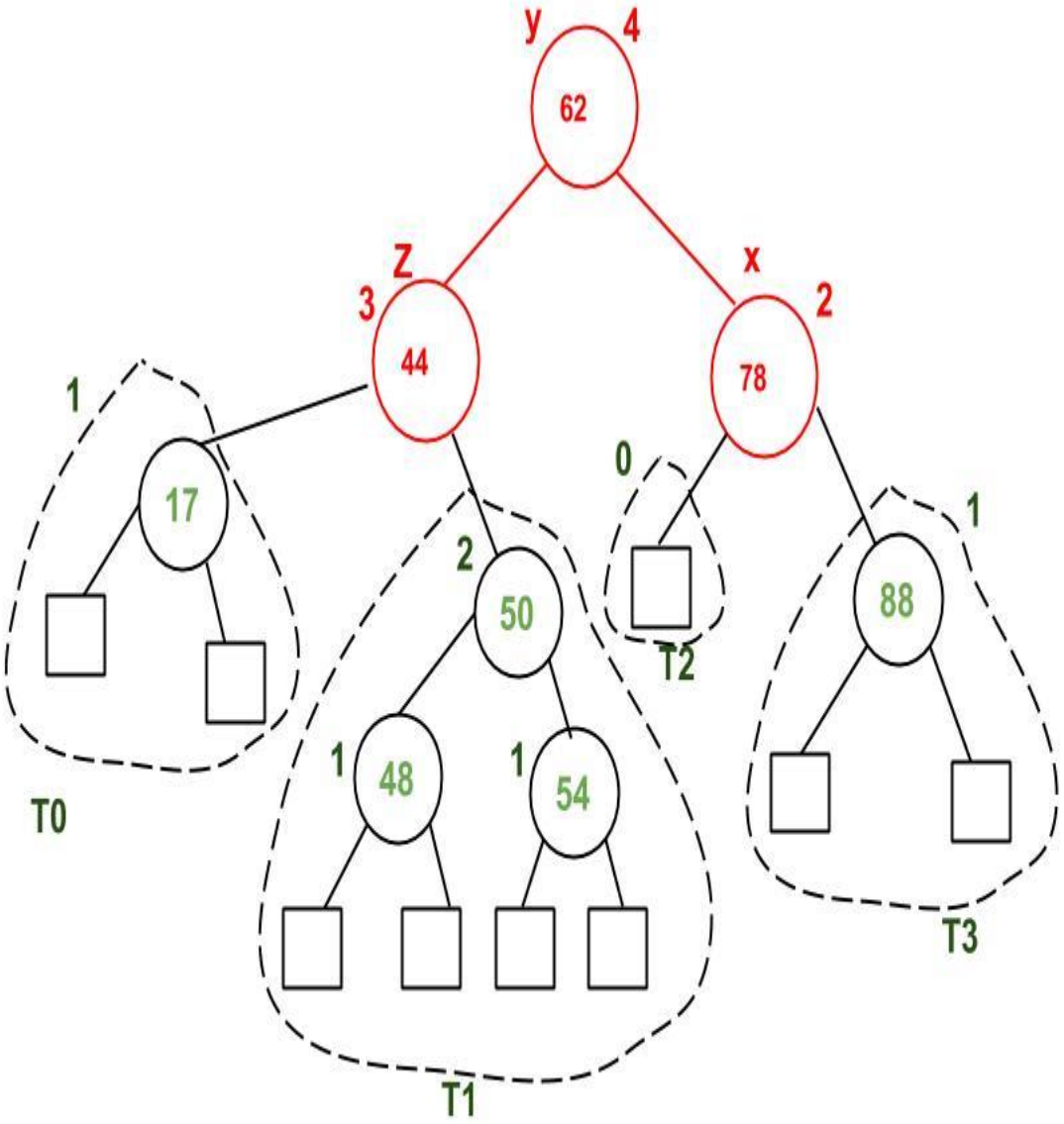
Unlike insertion, in deletion, after we perform a rotation at z, we may have to perform a rotation at ancestors of z. Thus, we must continue to trace the path until we reach the root.

Example:

MITRC

Example of deletion from an AVL Tree:




y
4
62
Z
3
44
x
2
78
1
17
2
50
0
1
88
1
48
1
54
T0
T1
T2
T3